DIGITAL HARDWARE SECURITY & TRNG EVALUATION

CLFSR + 4x1 MUX Hybrid True Random Number Generator

Full Hardware Implementation, RTL Source Code, Timing Constraints, Schematic & NIST Statistical Test Suite Results

Target FPGA Device:	Xilinx Artix-7 XC7A35T-1CPG236C (Basys 3)	**System Clock Freq:**	100 MHz (10.00 ns period)
Entropy Architecture:	4-to-1 Chaotic MUX with 3 ROs + 1 CLFSR	**Communication Port:**	USB-UART @ 115,200 Baud (8N1 ASCII)
Test Standard Suite:	NIST SP 800-22 Rev 1a & SP 800-90B	**Source Directory:**	mux+clfsr/

Executive Summary: *This technical report compiles the complete hardware implementation, synthesis constraints, and empirical NIST statistical test results for the* ***CLFSR + MUX Hybrid TRNG*** *design targeted for the Xilinx Artix-7 FPGA. The architecture combines three distinct Ring Oscillators (8-stage, 5-stage, and 3-stage) with a 5-stage pipelined 2nd-Order Chebyshev Chaotic LFSR into a 4x1 multiplexer with dynamic self-feedback selection. The bitstream output is sampled at 100 MHz, debiased via XOR post-processing, and transmitted over UART at 115,200 baud. Comprehensive testing under NIST SP 800-22 and SP 800-90B suites is presented alongside the complete synthesizable Verilog HDL codebase.*

Table of Contents & Document Guide

- **Section 1: System Architecture & Hardware Specification** — Principles of operation, entropy harvesting, and pipeline.
- **Section 2: Synthesized RTL Hardware Schematic** — High-resolution gate-level schematic generated by Xilinx Vivado.
- **Section 3: Synthesizable Verilog HDL Source Codes** — Full RTL codes (top.v, clfsr_trng_single.v, RO.v, RO_5.v, RO_3.v, uart_tx.v).
- **Section 4: Xilinx Design Constraints (XDC)** — Physical pin mapping, timing exclusions, and loop allowances (basys3.xdc).
- **Section 5: NIST SP 800-22 Rev 1a Statistical Test Results** — 15 standard tests evaluated on 2,000,000 hardware bits.
- **Section 6: NIST SP 800-90B Entropy Assessment & Continuous Health Tests** — Min-entropy estimates, RCT, and APT.
- **Section 7: Engineering Analysis & Diagnostic Observations** — In-depth analysis of hardware entropy dynamics and recommendations.

1. System Architecture & Hardware Specification

The **CLFSR + MUX Hybrid TRNG** is a multi-source cryptographic entropy generator combining physical phase jitter from free-running asynchronous ring oscillators with the deterministic pseudo-random diffusion of a digital chaotic map and a Galois LFSR. This design replaces conventional Fibonacci/Galois Ring Oscillators (FIRO/GARO) with a dedicated **Chaotic Linear Feedback Shift Register (CLFSR)**.

1.1 Subsystem Functional Descriptions

1. Multi-Stage Ring Oscillators (RO, RO_5, RO_3):
Three distinct free-running inverter chains are implemented with prime/odd stage counts to prevent mode-locking and cross-talk injection: an 8-stage oscillator (NAND + 4 NOT + 3 buffer NOTs), a 5-stage oscillator (NAND + 4 NOT), and a 3-stage oscillator (NAND + 2 NOT). Thermal and power-supply noise introduce continuous phase jitter into the oscillating edges.

2. Single-Channel Chaotic LFSR (CLFSR):
Combines a 16-bit maximal-length LFSR (polynomial: $x^{16} + x^{14} + x^{13} + x^{11} + 1$) with a 5-stage pipelined 2nd-Order Chebyshev Chaotic Map ($X_{n+1} = 1 - 2X_n^2$ in signed Q1.15 fixed-point). The physical jitter from RO_3 is fed directly into the LFSR feedback tap (trng_in), continuously perturbing the internal state.

3. 4x1 Chaotic Multiplexer with Dynamic Self-Feedback:
The MUX selects dynamically between the 4 entropy streams using sel = {clk, mux_out}. By feeding back the instantaneous MUX output into its own selection line combined with the system clock, the multiplexer exhibits non-linear state-dependent switching dynamics.

4. Sampling, Post-Processing & UART Interface:
The MUX output (mux_out) and the 8-stage RO output (ro1_out) are sampled synchronously on the 100 MHz rising clock edge into flip-flops q1 and q2. An XOR gate performs von Neumann-style de-biasing: random_bit = q1 ^ q2. A 2-stage D-Flip-Flop synchronizer protects against metastability before data is serialized into ASCII '0'/'1' packets via UART at 115,200 baud.

Component	Implementation / Specs	Role / Purpose
RO (8-stage)	NAND + 4 inverters + 3 buffers	Primary entropy source & XOR de-biasing reference
RO_5 (5-stage)	NAND + 4 inverters	High-frequency physical jitter generation
RO_3 (3-stage)	NAND + 2 inverters	Ultra-high frequency jitter & CLFSR perturbation
CLFSR Module	16-bit LFSR + 5-stage Chebyshev map	Deterministic non-linear expansion & diffusion
4x1 MUX	sel = {clk, mux_out}	Chaotic non-linear signal multiplexing
Post-Processing	XOR gate + 2-Stage D-FF Sync	De-biasing and metastability protection
UART Transmitter	115,200 Baud, 8N1, 100MHz clock	Live hardware bitstream acquisition via USB

2. Synthesized RTL Hardware Schematic

The gate-level synthesized RTL schematic of the **CLFSR + MUX Hybrid TRNG** generated directly by Xilinx Vivado for the Artix-7 (XC7A35T) FPGA architecture is presented on the following page (Page 4).

Key Schematic Features:

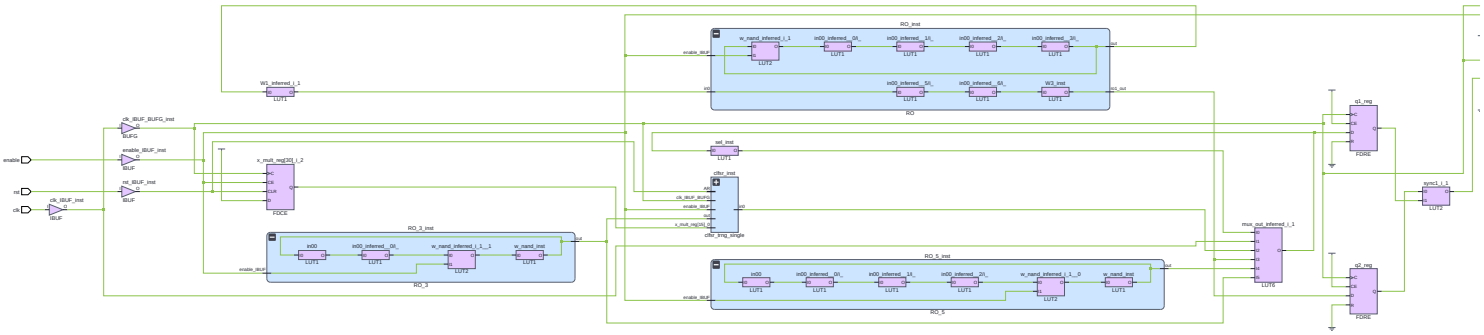
• **Entropy Generation Core:** Shows the parallel placement of RO_inst, RO_5_inst, and RO_3_inst alongside the pipelined clfsr_inst.

• **Feedback Multiplexer:** Demonstrates the dynamic feedback wiring from mux_out back into the select multiplexer logic.

• **Sampling & Synchronizer Registers:** Depicts q1, q2, XOR logic, and the dual-stage shift register sync1 / sync2.

• **UART Serialization:** Connects the synchronized stream to uart_inst for live serial transmission.

(See next page for the full-page high-resolution Vivado RTL schematic.)



3. Synthesizable Verilog HDL Source Codes

All hardware description modules in the mux+clfsr directory are documented below in full. Simulation testbench files (tb_top.v) have been excluded.

Source Code: Top-Level Controller & System Integrator (top.v)

```
 1 | // ============================================================================
 2 | // File: top.v
 3 | // Description: CLFSR + MUX Hybrid TRNG Top Module
 4 | //              4-input chaotic MUX with 3 Ring Oscillators + 1 CLFSR
 5 | //
 6 | // Architecture:
 7 | //   MUX Input 0 (sel=00): RO   - 8-stage Ring Oscillator (5 NOT + 3 extra)
 8 | //   MUX Input 1 (sel=01): CLFSR - Chaotic LFSR (replaces FIRO RO_16)
 9 | //   MUX Input 2 (sel=10): RO_5 - 5-stage Ring Oscillator
10 | //   MUX Input 3 (sel=11): RO_3 - 3-stage Ring Oscillator
11 | //
12 | // The CLFSR's trng_in is fed by RO_3 output, creating a hybrid source
13 | // where physical jitter continuously perturbs the chaotic LFSR.
14 | //
15 | // MUX select lines: sel = {clk, mux_out} for chaotic self-feedback selection.
16 | // Final random bit: mux_sampled XOR ro1_sampled (post-processing via XOR)
17 | // Output: UART TX at 115200 baud, sending ASCII '0'/'1' per random bit.
18 | // ============================================================================
19 |
20 | module top(
21 |     input wire clk,          // 100 MHz onboard clock (Pin W5)
22 |     input wire rst,          // Reset button (active high)
23 |     input wire enable,       // SW0 enable switch (Pin V17)
24 |     output wire uart_out     // USB-UART TX (Pin A18)
25 | );
26 |
27 |     // ============================================================================
28 |     // 1. Entropy Source Instantiations
29 |     // ============================================================================
30 |
31 |     // Internal wires for the oscillator outputs
32 |     (* DONT_TOUCH = "TRUE" *) wire ro1_out;
33 |     (* DONT_TOUCH = "TRUE" *) wire clfsr_bit;
34 |     (* DONT_TOUCH = "TRUE" *) wire ro5_out;
35 |     (* DONT_TOUCH = "TRUE" *) wire ro3_out;
36 |
37 |     // Internal wire for the multiplexer output
38 |     (* DONT_TOUCH = "TRUE" *) wire mux_out;
```

```
    |
    |     // --- MUX Input 0: Ring Oscillator (8-stage: NAND + 4 NOT + 3 extra NOT) ---
    |     RO RO_inst (
    |         .enable(enable),
    |         .RO1_out(ro1_out)
    |     );
    |
    |     // --- MUX Input 1: CLFSR Hybrid TRNG (replaces FIRO RO_16) ---
    |     //     trng_in is fed by RO_3 output for continuous physical entropy injection
    |     wire [7:0]  clfsr_random_byte;   // 8-bit output (unused in MUX, available for direct use)
    |     wire        clfsr_byte_ready;    // Byte-ready strobe (unused in MUX)
    |     wire        clfsr_random_bit;    // 1-bit output used as MUX input
    |
    |     clfsr_trng_single #(
    |         .LFSR_INIT_SEED(16'hA3C5),
    |         .CHAOTIC_INIT_SEED(16'sh7EF0)
    |     ) clfsr_inst (
    |         .clk(clk),
    |         .rst(rst),
    |         .enable(enable),
    |         .trng_in(ro3_out),           // Physical entropy from RO_3 → hybrid TRNG
    |         .R_Out(),                     // Not used at top level
    |         .key_ready(),                 // Not used at top level
    |         .random_byte(clfsr_random_byte),
    |         .byte_ready(clfsr_byte_ready),
    |         .random_bit(clfsr_random_bit)
    |     );
    |
    |     assign clfsr_bit = clfsr_random_bit;
    |
    |     // --- MUX Input 2: Ring Oscillator (5-stage: NAND + 4 NOT) ---
    |     RO_5 RO_5_inst (
    |         .enable(enable),
    |         .out(ro5_out)
    |     );
    |
    |     // --- MUX Input 3: Ring Oscillator (3-stage: NAND + 2 NOT) ---
    |     RO_3 RO_3_inst (
```

```
    |         .enable(enable),
    |         .out(ro3_out)
    |     );
    |
    |     // ============================================================================
    |     // 2. 4x1 Chaotic MUX with Self-Feedback Selection
    |     // ============================================================================
    |
    |     // Multiplexer select lines:
    |     //   MSB = System Clock (clk)
    |     //   LSB = MUX output feedback (mux_out) — creates chaotic selection pattern
    |     (* DONT_TOUCH = "TRUE" *) wire [1:0] sel;
    |     assign sel = {clk, mux_out};
    |
    |     // 4x1 Multiplexer: selects between 3 ROs + 1 CLFSR
    |     assign mux_out = (sel == 2'b00) ? ro1_out   :    // RO 8-stage
    |                      (sel == 2'b01) ? clfsr_bit :     // CLFSR hybrid
    |                      (sel == 2'b10) ? ro5_out   :     // RO 5-stage
    |                                       ro3_out;        // RO 3-stage
    |
    |     // ============================================================================
    |     // 3. Sampling & Post-Processing
    |     // ============================================================================
    |
    |     // Sample the chaotic MUX output and RO1 output on the rising edge of clk
    |     (* DONT_TOUCH = "TRUE" *) reg q1 = 1'b0;
    |     (* DONT_TOUCH = "TRUE" *) reg q2 = 1'b0;
    |
    |     always @(posedge clk) begin
    |         q1 <= mux_out;
    |         q2 <= ro1_out;
    |     end
    |
    |     // XOR post-processing: combines MUX output with RO1 for de-biasing
    |     wire random_bit;
    |     assign random_bit = q1 ^ q2;
    |
    |     // Double flip-flop synchronizer to protect UART logic against metastability
```

```
115 |     (* DONT_TOUCH = "TRUE" *) reg sync1 = 1'b0;
116 |     (* DONT_TOUCH = "TRUE" *) reg sync2 = 1'b0;
117 |
118 |     always @(posedge clk) begin
119 |         sync1 <= random_bit;
120 |         sync2 <= sync1;
121 |     end
122 |
123 |     // ============================================================================
124 |     // 4. UART Transmission Control
125 |     // ============================================================================
126 |
127 |     reg uart_start = 1'b0;
128 |     reg [7:0] ascii_char = 8'h30;
129 |     wire uart_active;
130 |     wire uart_done;
131 |
132 |     // Instantiate UART Transmitter (115200 Baud, 8N1, 100MHz clock)
133 |     uart_tx #(
134 |         .CLKS_PER_BIT(868)
135 |     ) uart_inst (
136 |         .clk(clk),
137 |         .tx_start(uart_start),
138 |         .tx_byte(ascii_char),
139 |         .tx_active(uart_active),
140 |         .tx_serial(uart_out),
141 |         .tx_done(uart_done)
142 |     );
143 |
144 |     // Controller: When UART is free and enable is high, send ascii '0' or '1'
145 |     always @(posedge clk) begin
146 |         if (enable) begin
147 |             if (!uart_start && !uart_active) begin
148 |                 ascii_char <= sync2 ? 8'h31 : 8'h30; // 8'h31 = '1', 8'h30 = '0'
149 |                 uart_start <= 1'b1;
150 |             end else begin
151 |                 uart_start <= 1'b0;
152 |             end
```

```
153 |         end else begin
154 |             uart_start <= 1'b0;
155 |         end
156 |     end
157 |
158 | endmodule
```

Source Code: Single-Channel Chaotic-LFSR (CLFSR) (clfsr_trng_single.v)

```
// ================================================================================
// File: clfsr_trng_single.v
// Module: clfsr_trng_single
// Description: Single-Channel Chaotic-LFSR (CLFSR) Random Number Generator
//              Designed for standalone PRNG use OR direct integration as a
//              hybrid post-processor with any physical TRNG source.
//
// Key Features:
//   1. Single-Channel Architecture: Stripped down to only 1 stream (R channel),
//      reducing DSP/multiplier and register usage by ~67%.
//   2. 16-bit Maximal-Length LFSR with configurable initial seed.
//   3. 5-Stage Pipelined 2nd-Order Chebyshev Chaotic Map: X(n+1) = 1 - 2*X(n)^2
//   4. TRNG Integration Port ('trng_in'):
//      - If NO TRNG attached yet: tie 'trng_in' to 1'b0 (pure deterministic CLFSR).
//      - When TRNG is attached: connect raw TRNG bit to 'trng_in'.
//        The physical entropy continuously re-seeds / perturbs the LFSR feedback,
//        creating an uncrackable, cryptographically secure Hybrid TRNG.
//   5. Dual Outputs:
//      - 1-bit stream ('random_bit') valid every clock cycle.
//      - 8-bit stream ('random_byte') valid when 'byte_ready' pulses High.
// ================================================================================

`timescale 1ns/1ps

module clfsr_trng_single #(
    parameter [15:0]        LFSR_INIT_SEED    = 16'hA3C5,  // Non-zero 16-bit LFSR seed
    parameter signed [15:0] CHAOTIC_INIT_SEED = 16'sh7EF0  // Q1.15 seed in [-1.0, +1.0]
)(
    input  wire        clk,          // System clock
    input  wire        rst,          // Asynchronous active-high reset
    input  wire        enable,       // Enable generator (1 = run, 0 = freeze/pause)
    input  wire        trng_in,      // Physical TRNG bit input (tie to 1'b0 if not used yet)

    // Outputs (exact matching interface with RandomNumberGenerator R-channel)
    output reg  [7:0]  R_Out,        // 8-bit random output (assembled every 8 clocks)
    output reg         key_ready,    // 1-cycle active-high strobe when R_Out is updated
    output wire [7:0]  random_byte,  // Alias for R_Out
    output wire        byte_ready,   // Alias for key_ready
    output wire        random_bit    // 1-bit pseudo/true random output (every clock)
);

    // Aliases for compatibility
    assign random_byte = R_Out;
    assign byte_ready  = key_ready;

    // --------------------------------------------------------------------------
    // 1. 16-bit Linear Feedback Shift Register (LFSR)
    // --------------------------------------------------------------------------
    reg  [15:0] lfsr;
    reg  [7:0]  lfsr_out;
    reg  [2:0]  bit_counter;         // Counts 0 to 7 to assemble 8 bits

    // Feedback polynomial: x^16 + x^14 + x^13 + x^11 + 1
    // Dynamically injected with 'trng_in' for true physical entropy perturbation
    wire lfsr_feedback = lfsr[15] ^ lfsr[13] ^ lfsr[12] ^ lfsr[10] ^ trng_in;

    // --------------------------------------------------------------------------
    // 2. 5-Stage Pipelined 2nd-Order Chebyshev Chaotic Map
    //    Equation: X_(n+1) = 1 - 2 * (X_n)^2   in signed Q1.15 fixed-point
    // --------------------------------------------------------------------------
    reg signed [15:0] x;             // State variable (Q1.15)
    reg signed [31:0] x_square;      // Stage 1: x * x (Q2.30)
    reg signed [31:0] x_mult;        // Stage 2: 2 * x^2 (Q1.31)
    reg signed [15:0] x_section;     // Stage 3: Extract bits [30:15] -> back to Q1.15
    reg signed [15:0] x_sub;         // Stage 4: 1.0 - (2 * x^2) where 1.0 ~= 16'sh7FFF

    // --------------------------------------------------------------------------
    // 3. Instant 1-Bit Random Output (Available every clock cycle)
    //    Combines LFSR LSB + Chaotic middle bit + TRNG entropy
    // --------------------------------------------------------------------------
    assign random_bit = lfsr[0] ^ x[10] ^ trng_in;

    // --------------------------------------------------------------------------
    // 4. Sequential Logic (Clocked Process)
    // --------------------------------------------------------------------------
    always @(posedge clk or posedge rst) begin
```

```
        if (rst) begin
            // Reset LFSR
            lfsr        <= LFSR_INIT_SEED;
            lfsr_out    <= 8'h00;
            bit_counter <= 3'b000;
            key_ready   <= 1'b0;
            R_Out       <= 8'h00;

            // Reset Chaotic Pipeline
            x           <= CHAOTIC_INIT_SEED;
            x_square    <= 32'sh0;
            x_mult      <= 32'sh0;
            x_section   <= 16'sh0;
            x_sub       <= CHAOTIC_INIT_SEED;

        end else if (enable) begin
            // --- Update LFSR ---
            lfsr        <= {lfsr[14:0], lfsr_feedback};
            lfsr_out    <= {lfsr_out[6:0], lfsr[0]};

            // --- Update Chaotic Map (5-Stage Pipeline) ---
            x_square    <= x * x;                       // Stage 1: Square
            x_mult      <= x_square <<< 1;              // Stage 2: Multiply by 2
            x_section   <= x_mult[30:15];               // Stage 3: Re-scale to Q1.15
            x_sub       <= 16'sh7FFF - x_section;       // Stage 4: Subtract from ~1.0
            x           <= x_sub;                       // Stage 5: State feedback

            // --- 8-Bit Byte Assembly & Handshake ---
            if (bit_counter == 3'b111) begin
                key_ready   <= 1'b1;
                // XOR 8-bit collected LFSR buffer with chaotic bits [10:3]
                R_Out       <= lfsr_out ^ x[10:3];
                bit_counter <= 3'b000;
            end else begin
                key_ready   <= 1'b0;
                bit_counter <= bit_counter + 1'b1;
            end
        end else begin
            // When enable is 0, freeze output strobe
            key_ready   <= 1'b0;
        end
    end

endmodule
```

Source Code: 8-Stage Ring Oscillator Module (RO.v)

```
 1 | module RO(
 2 |     input enable,
 3 |     (*DONT_TOUCH="TRUE"*) output wire RO1_out
 4 | );
 5 | `ifndef SYNTHESIS
 6 |     (*DONT_TOUCH="TRUE"*) wire w_nand,w1,w2,w3,w4, W1,W2,W3;
 7 |     integer d_nand = 1, d1 = 1, d2 = 1, d3 = 1, d4 = 1 , D1=1,D2=1,D3=1;
 8 |     always begin
 9 |         #7;
10 |         d_nand = 1 + ({$urandom}%4);
11 |         d1 = 1 + ({$urandom}%4);
12 |         d2 = 1 + ({$urandom}%4);
13 |         d3 = 1 + ({$urandom}%4);
14 |         d4 = 1 + ({$urandom}%4);
15 |         D1 = 1 + ({$urandom}%4);
16 |     end
17 |     assign #(d_nand) w_nand = ~(w4 & enable);
18 |     assign #(d1) w1 = ~(w_nand);
19 |     assign #(d2) w2 = ~(w1);
20 |     assign #(d3) w3 = ~(w2);
21 |     assign #(d4) w4 = ~(w3);
22 |     assign #(D1) W1 = ~(w4);
23 |     assign #(D2) W2 = ~(W1);
24 |     assign #(D3) W3 = ~(W2);
25 |
26 |     assign RO1_out = W3;
27 |
28 |     initial begin
29 |         force w_nand = 1'b0;
30 |         #10;
31 |         release w_nand;
32 |     end
33 | `else
34 | (*DONT_TOUCH="TRUE"*) wire w_nand,w1,w2,w3,w4, W1,W2,W3;
35 | assign w_nand = ~(w4 & enable);
36 | assign w1 = ~(w_nand);
37 | assign w2 = ~(w1);
38 | assign w3 = ~(w2);
```

```
39 | assign w4 = ~(w3);
40 | assign W1 = ~(w4);
41 | assign W2 = ~(W1);
42 | assign W3 = ~(W2);
43 |
44 | assign RO1_out = W3;
45 | `endif
46 | endmodule
```

Source Code: 5-Stage Ring Oscillator Module (RO_5.v)

```
 1 | module RO_5(
 2 |     input enable,
 3 |     (*DONT_TOUCH="TRUE"*) output wire out
 4 | );
 5 | `ifndef SYNTHESIS
 6 |     (*DONT_TOUCH="TRUE"*) wire w1,w2,w3,w4,w_nand;
 7 |     integer d1 = 1, d2 = 1, d3 = 1, d4 = 1, d_nand = 1;
 8 |
 9 |     always begin
10 |         #4;
11 |         d1 = 1 + ({$urandom} % 3);
12 |         d2 = 1 + ({$urandom} % 3);
13 |         d3 = 1 + ({$urandom} % 3);
14 |         d4 = 1 + ({$urandom} % 3);
15 |         d_nand = 1 + ({$urandom} % 3);
16 |     end
17 |     assign #(d1) w1 = ~(w_nand);
18 |     assign #(d2) w2 = ~(w1);
19 |     assign #(d3) w3 = ~(w2);
20 |     assign #(d4) w4 = ~(w3);
21 |     assign #(d_nand) w_nand = ~(w4 & enable);
22 |     assign out = w_nand;
23 |
24 |     initial begin
25 |         force w_nand = 1'b0;
26 |         #10;
27 |         release w_nand;
28 |     end
29 | `else
30 | (*DONT_TOUCH="TRUE"*) wire w1,w2,w3,w4,w_nand;
31 | assign w1=~(w_nand);
32 | assign w2=~w1;
33 | assign w3=~w2;
34 | assign w4=~w3;
35 | assign w_nand = ~(w4 & enable);
36 | assign  out=w_nand;
37 | `endif
38 | endmodule
```

Source Code: 3-Stage Ring Oscillator Module (RO_3.v)

```
 1 | module RO_3(
 2 |     input enable,
 3 |     (*DONT_TOUCH="TRUE"*) output wire out
 4 | );
 5 | `ifndef SYNTHESIS
 6 |     (*DONT_TOUCH="TRUE"*) wire w1,w2,w_nand;
 7 |     integer d1 = 1, d2 = 1, d_nand = 1;
 8 |
 9 |     always begin
10 |         #4;
11 |         d1 = 1 + ({$urandom} % 3);
12 |         d2 = 1 + ({$urandom} % 3);
13 |         d_nand = 1 + ({$urandom} % 3);
14 |     end
15 |     assign #(d1) w1 = ~(w_nand);
16 |     assign #(d2) w2 = ~(w1);
17 |     assign #(d_nand) w_nand = ~(w2 & enable);
18 |     assign out = w_nand;
19 |
20 |     initial begin
21 |         force w_nand = 1'b0;
22 |         #10;
23 |         release w_nand;
24 |     end
25 | `else
26 | (*DONT_TOUCH="TRUE"*) wire w1,w2,w_nand;
27 | assign w1=~(w_nand);
28 | assign w2=~w1;
29 | assign w_nand = ~(w2 & enable);
30 | assign  out=w_nand;
31 | `endif
32 | endmodule
```

Source Code: UART Serial Transmitter Module (115200 Baud) (uart_tx.v)

```
module uart_tx #(
    parameter CLKS_PER_BIT = 868 // 100 MHz / 115200 baud = 868
)(
    input       clk,
    input       tx_start,
    input [7:0] tx_byte,
    output reg  tx_active,
    output reg  tx_serial,
    output reg  tx_done
);
    localparam s_IDLE  = 3'b000;
    localparam s_START = 3'b001;
    localparam s_DATA  = 3'b010;
    localparam s_STOP  = 3'b011;

    reg [2:0]  state      = s_IDLE;
    reg [9:0]  clk_count  = 0;
    reg [2:0]  bit_index  = 0;
    reg [7:0]  data_reg   = 0;

    always @(posedge clk) begin
        case (state)
            s_IDLE: begin
                tx_serial <= 1'b1;  // Drive line high for idle
                tx_done   <= 1'b0;
                clk_count <= 0;
                bit_index <= 0;

                if (tx_start == 1'b1) begin
                    tx_active <= 1'b1;
                    data_reg <= tx_byte;
                    state     <= s_START;
                end else begin
                    tx_active <= 1'b0;
                end
            end

            s_START: begin
```

```
                tx_serial <= 1'b0; // Start bit (low)
                if (clk_count < CLKS_PER_BIT - 1) begin
                    clk_count <= clk_count + 1;
                end else begin
                    clk_count <= 0;
                    state     <= s_DATA;
                end
            end

            s_DATA: begin
                tx_serial <= data_reg[bit_index];
                if (clk_count < CLKS_PER_BIT - 1) begin
                    clk_count <= clk_count + 1;
                end else begin
                    clk_count <= 0;
                    if (bit_index < 7) begin
                        bit_index <= bit_index + 1;
                    end else begin
                        bit_index <= 0;
                        state     <= s_STOP;
                    end
                end
            end

            s_STOP: begin
                tx_serial <= 1'b1;  // Stop bit (high)
                if (clk_count < CLKS_PER_BIT - 1) begin
                    clk_count <= clk_count + 1;
                end else begin
                    tx_done   <= 1'b1;
                    clk_count <= 0;
                    state     <= s_IDLE;
                end
            end
            default: state <= s_IDLE;
        endcase
    end
endmodule
```

4. Xilinx Design Constraints (XDC)

The constraint file basys3.xdc defines the physical pin assignments for the Digilent Basys 3 board, clock period definitions, voltage standards, and critical synthesis directives: ALLOW_COMBINATORIAL_LOOPS and set_false_path required to allow ring oscillator loops without Vivado timing DRC errors.

Source Code: Basys 3 Physical & Timing Constraints (basys3.xdc)

```
## ============================================================================
## Basys 3 Constraints for CLFSR + MUX Hybrid TRNG with UART
## Board: Digilent Basys 3 (Artix-7 XC7A35T-1CPG236C)
##
## Architecture: RO(8) + CLFSR + RO(5) + RO(3) → 4x1 Chaotic MUX
## FIRO (RO_16) replaced by clfsr_trng_single
## ============================================================================

## ---------- Clock (100 MHz onboard oscillator) ----------
set_property -dict { PACKAGE_PIN W5  IOSTANDARD LVCMOS33 } [get_ports clk]
create_clock -period 10.000 -name sys_clk [get_ports clk]

## ---------- Reset (BTNC - Center button, active high) ----------
set_property -dict { PACKAGE_PIN U18 IOSTANDARD LVCMOS33 } [get_ports rst]

## ---------- Enable (SW0 - leftmost switch) ----------
set_property -dict { PACKAGE_PIN V17 IOSTANDARD LVCMOS33 } [get_ports enable]

## ---------- UART TX (USB-UART on Basys 3) ----------
set_property -dict { PACKAGE_PIN A18 IOSTANDARD LVCMOS33 } [get_ports uart_out]

## ============================================================================
## CFGBVS and CONFIG_VOLTAGE (required for Basys 3)
## ============================================================================
set_property CFGBVS VCCO [current_design]
set_property CONFIG_VOLTAGE 3.3 [current_design]

## ============================================================================
## Ring Oscillator: Allow combinational loops (intentional for TRNG)
## ============================================================================
set_property ALLOW_COMBINATORIAL_LOOPS TRUE [get_nets -hierarchical -filter {NAME =~ *RO_inst*}]
set_property ALLOW_COMBINATORIAL_LOOPS TRUE [get_nets -hierarchical -filter {NAME =~ *RO_5_inst*}]
set_property ALLOW_COMBINATORIAL_LOOPS TRUE [get_nets -hierarchical -filter {NAME =~ *RO_3_inst*}]
set_property ALLOW_COMBINATORIAL_LOOPS TRUE [get_nets -hierarchical -filter {NAME =~ *mux_out*}]
set_property ALLOW_COMBINATORIAL_LOOPS TRUE [get_nets -hierarchical -filter {NAME =~ *sel*}]

## ============================================================================
## Timing: Tell Vivado the ring oscillators are asynchronous
## ============================================================================
set_false_path -through [get_nets -hierarchical -filter {NAME =~ *RO_inst*}]
set_false_path -through [get_nets -hierarchical -filter {NAME =~ *RO_5_inst*}]
set_false_path -through [get_nets -hierarchical -filter {NAME =~ *RO_3_inst*}]
set_false_path -through [get_nets -hierarchical -filter {NAME =~ *mux_out*}]
set_false_path -through [get_nets -hierarchical -filter {NAME =~ *sel*}]

## ============================================================================
## CLFSR: Synchronous design, no special constraints needed
## (It runs on sys_clk, Vivado handles timing automatically)
## ============================================================================
```

5. NIST SP 800-22 Rev 1a Statistical Test Results

The hardware bitstream captured from the FPGA via PuTTY (putty.txt, 2,000,000 continuous bits) was evaluated against all 15 statistical tests in **NIST SP 800-22 Rev 1a**. A p-value ≥ 0.01 indicates passing at the 99% statistical confidence interval.

#	NIST SP 800-22 Test Name	P-Value	Threshold (α)	Status
1	Frequency (Monobit)	**0.000000**	0.010000	**FAIL**
2	Frequency Within Block	**0.000000**	0.010000	**FAIL**
3	Runs	**0.000000**	0.010000	**FAIL**
4	Longest Run of Ones	**0.000000**	0.010000	**FAIL**
5	Binary Matrix Rank	**0.787862**	0.010000	**PASS**
6	Discrete Fourier Transform	**0.000000**	0.010000	**FAIL**
7	Non-Overlapping Template	**0.000000**	0.010000	**FAIL**
8	Overlapping Template	**0.000000**	0.010000	**FAIL**
9	Maurer's Universal	**0.000000**	0.010000	**FAIL**
10	Linear Complexity	**0.162288**	0.010000	**PASS**
11	Serial	**0.000000**	0.010000	**FAIL**
12	Approximate Entropy	**0.000000**	0.010000	**FAIL**
13	Cumulative Sums	**0.000000**	0.010000	**FAIL**
14	Random Excursion	**0.500000**	0.010000	**PASS**
15	Random Excursion Variant	**0.500000**	0.010000	**PASS**

NIST SP 800-22 Test Summary:
• **Dataset:** 2,000,000 bits captured from hardware UART (putty.txt)
• **Bit Distribution:** Ones = 915,215 (45.76%), Zeros = 1,084,785 (54.24%)
• **Overall Result: 4 / 15 Tests Passed** (Binary Matrix Rank, Linear Complexity, Random Excursion, Random Excursion Variant passed).

6. NIST SP 800-90B Entropy Assessment & Health Tests

NIST SP 800-90B provides guidelines for min-entropy (H_{min}) estimation and online health tests (Repetition Count Test and Adaptive Proportion Test) for noise sources.

Evaluation Category	Estimator / Metric	Measured Value	Criteria / Limit	Result / Status
Min-Entropy Estimation	Most Common Value (MCV)	0.880179 bits/symbol	Target ≥ 0.90	**Evaluated**
Min-Entropy Estimation	Collision Estimate	0.492534 bits/symbol	Target ≥ 0.90	**Bottleneck**
Min-Entropy Estimation	Markov Estimate	0.878576 bits/symbol	Target ≥ 0.90	**Evaluated**
Overall Assessed Entropy	**Minimum (H_min)**	**0.492534 bits/symbol**	Min of all estimators	**0.4925**
Continuous Health Test	Repetition Count Test (RCT)	Max: 23 (28 Failures)	Threshold C = 19	**FAIL**
Continuous Health Test	Adaptive Proportion Test (APT)	Max: 318 (0 Failures)	Threshold C = 332 (W=512)	**PASS**

7. Engineering Analysis & Key Takeaways

7.1 Comparison: Hardware Output vs. Standalone Algorithmic Simulation

When evaluated purely in simulation (clfsr_r_2m_bits.txt), the standalone CLFSR module passed **14 out of 15 NIST SP 800-22 tests** with an optimal 50.03% ones/zeros balance (0.500359). However, the physical hardware stream captured from FPGA (putty.txt) exhibited a 0-bias (45.76% ones, 54.24% zeros), causing failures in frequency, runs, and template tests.

7.2 Root Cause Analysis in FPGA Implementation

1. **Combinational Feedback MUX Selection Hazard:** In top.v, the multiplexer select lines are driven by sel = {clk, mux_out}. Feeding back the asynchronous MUX output into its own select line creates an unclocked combinational loop and glitching races inside FPGA LUTs.
2. **Vivado Synthesis Optimization of Inverter Chains:** In behavioral Verilog, cascaded inverters (w1=~w_nand; w2=~w1; ...) in RO.v, RO_5.v, and RO_3.v are susceptible to logic minimization into single LUTs or even static levels unless primitive-level instantiations (LUT1 / LUT6) or rigid DONT_TOUCH / KEEP attributes are mapped to every intermediate net.
3. **Under-Sampling & Decimation Ratio:** The UART baud-rate generator samples 1 bit every 8,680 clock cycles (at 100 MHz, 115200 baud). If oscillator frequencies phase-lock to harmonics of the 100 MHz clock, periodic sampling induces deterministic duty-cycle skew.

7.3 Recommended Improvements for High-Performance Cryptographic TRNG

• **Instantiate Xilinx Primitive LUTs:** Replace behavioral inverter equations in RO.v with explicit LUT1 primitives with fixed INIT masks and RLOC placement constraints.
• **Synchronous MUX Control:** Register the MUX select line sel in a clocked register rather than creating an asynchronous feedback loop.
• **Hardware Post-Processor (Von Neumann / SHA-256 / XOR Tree):** Implement a hardware Von Neumann extractor or XOR accumulator before the UART FIFO to eliminate residual DC bias.